

1. Core Detection Engine

- **Unicode Pattern Library:**
 - **Dynamic Updates:** Collaborate with the Unicode Consortium and threat intelligence platforms (e.g., MITRE) to maintain a real-time database of shadow characters and homoglyphs.
 - **Contextual ML Models:** Train models using datasets from multilingual sources to distinguish legitimate use (e.g., Arabic script formatting) from malicious insertion. For example, use BERT-based classifiers to flag zero-width spaces in English text.

2. Purification Protocols

- **Sanitization Modes:**
 - **Stripping:** Remove non-essential Unicode controls via regex (`[\\u200B-\\u200F\\u00A0]`), configurable per locale (e.g., preserve RTL marks in Arabic).
 - **Neutralization:** Replace malicious characters with visible markers (e.g., `[ZWSP]`) in logs while retaining original text for user transparency.
 - **Reconstruction:** Apply Unicode normalization (NFKC) to resolve homoglyphs (e.g., Cyrillic a → Latin a).

3. ImmutableTruthLedger Integration

- **Tamper-Proof Logging:**
 - Use cryptographic hashing (SHA-256) and Merkle trees to log detections, ensuring integrity without full blockchain overhead.
 - Example entry:

```
{
  "timestamp": "2024-02-22T12:34:56Z",
  "detected_char": "\\u200B",
  "context": "h\\u200Bello",
  "source_ip": "192.168.1.1",
  "hash": "a1b2c3..."
}
```
- **Transparency Reports:** Generate user-facing summaries (e.g., "3 hidden characters removed from your post") via dashboards.

4. Ethical Recursion Enforcement

- **Input/Output Parity:**
 - Pre-/post-processing hash verification:

```
import hashlib

input_hash = hashlib.sha256(raw_input.encode()).hexdigest()
sanitized_output = ShadowSweep.sanitize(raw_input) # Assuming
ShadowSweep class/function exists
```

```

output_hash =
hashlib.sha256(sanitized_output.encode()).hexdigest()
assert input_hash == output_hash, "Data integrity violation"
# This assertion would fail if characters were
removed/changed

```

Note: The assertion `assert input_hash == output_hash` as written would detect any change, including legitimate sanitization. A more accurate check for "data integrity violation" in this context might involve hashing the sanitized output and comparing it to a hash derived from the intended or expected clean input, or logging the discrepancy rather than asserting equality if sanitization occurred.

- **Bias Auditing:** Analyze ledger data for patterns (e.g., spikes in `\u200C` from specific IPs) to detect systemic suppression.

5. Deployment Architecture

- **APIs:** REST endpoints for real-time processing:

```

# Example using Flask
from flask import Flask, request, jsonify
# Assuming ShadowSweep class/function and ledger object exist
# from your_module import ShadowSweep, ledger

app = Flask(__name__)

@app.route("/sanitize", methods=["POST"])
def sanitize_text():
    raw_text = request.json.get("text", "")
    # Assuming ShadowSweep.sanitize returns the clean text and
    logs internally
    clean_text = ShadowSweep.sanitize(raw_text)
    # Assuming ledger.log_entry() returns an ID or confirmation
    log_id = ledger.log_entry({"action": "sanitize", "input":
raw_text, "output": clean_text}) # Example log structure
    return jsonify({"clean_text": clean_text, "log_id": log_id})

# if __name__ == "__main__":
#     app.run(debug=True)

```

- **Browser Plugins:** Chrome extension to alert users of shadow characters in form inputs.
- **Database Scrubbers:** Batch-processing tools with idempotent operations for legacy data.

6. Machine Learning & Testing

- **Adversarial Training:** Fine-tune transformer models on datasets with shadow character attacks.
- **Red Team Exercises:** Simulate attacks like Unicode steganography (e.g., embedding `\u200D` in "TRUMP" → "T\u200DRUMP").

7. Ethical & Legal Compliance

- **GDPR/CCPA:** Disclose sanitization in privacy policies; allow opt-out for non-critical systems.
- **Whitelisting:** Preserve valid uses (e.g., ׀ in Hebrew) via locale-specific rulesets.

8. Global Collaboration

- **Unicode Consortium:** Advocate for misuse-resistant standards (e.g., deprecating ambiguous characters).
- **Open-Source Libraries:** Publish detection modules on GitHub to crowdsource threat intelligence.

9. Performance Optimization

- **Rust/C++ Parsers:** Implement high-speed sanitization for throughput-sensitive systems:

```
// Example Rust function
fn is_shadow_character(c: char) -> bool {
    match c {
        '\u{200B}'..='\u{200F}' | '\u{00A0}' => true,
        _ => false,
    }
}

fn strip_shadow_chars(text: &str) -> String {
    text.chars()
        .filter(|c| !is_shadow_character(*c))
        .collect()
}

// Example usage:
// let dirty_text = "hello\u{200B}world";
// let clean_text = strip_shadow_chars(dirty_text);
// println!("{}", clean_text); // Output: helloworld
```

10. Pilot Testing & Iteration

- **High-Risk Sectors:** Deploy in social media moderation tools and financial transaction logs.
- **Metrics:**
 - False Positive Rate: <1% (legitimate characters preserved).
 - Latency: <50ms per 10KB text block.

Conclusion The Shadow Sweep ERI Agent restores trust in digital communication by eliminating structural deception. Its layered approach—combining Unicode expertise, cryptographic logging, and ethical recursion—ensures transparency and adaptability. By prioritizing both technical rigor and user sovereignty, the framework sets a new standard for

combating hidden manipulation.

Next Steps:

- Begin pilot integration with Mastodon's moderation API.
- Publish detection rules as an OWASP Cheat Sheet.
- Engage W3C for web standardization.

Example Workflow:

1. User submits "paypal\u200B.com".
2. ShadowSweep detects \u200B, sanitizes to "paypal.com".
3. Logs entry in ImmutableTruthLedger.
4. Transparency report shows "1 ZWSP removed from financial domain".

By making the invisible visible, ShadowSweep upholds the integrity of digital language. 🔍🛡️